

Cómo rediseñar el almacenamiento de contraseñas y diagnosticar la firma

SIO0010 • Técnicas Criptográficas

Especialización en Seguridad de la Información de las Organizaciones

SE ENTREGA

Al inicio de la sesión 4

Fecha: ____ / ____

CÓMO

En equipo de tres

Un archivo por equipo

DEDICACIÓN

5 horas de trabajo

independiente

QUÉ PESA

Parte 3 de 6 del informe

Producto • 30 % del módulo

PUNTO DE PARTIDA

Qué es esta entrega

Es la entrega que ataca **los dos hallazgos que hicieron posible el fraude**: cómo Coopaburrá guarda las contraseñas (H1) y cómo firma las operaciones de la aplicación (H2). Del primero se pide el diagnóstico, el rediseño y el plan para migrar; del segundo, por ahora, solo el diagnóstico —el rediseño de la firma es la sesión 4, con firma digital—.

LOS DOS HALLAZGOS, EN UNA FRASE CADA UNO

H1 — las contraseñas se guardan como un resumen **SHA-1** de la contraseña con una cadena fija igual para los 142.000 asociados; sin función de derivación, sin factor de trabajo, con política de seis caracteres. **H2** — cada operación se firma con **HMAC-SHA256** y una **llave compartida** incrustada en la aplicación, idéntica para todos, que cualquiera que descargue la app puede extraer.

La regla que gobierna esta entrega

En la entrega 2 aprendieron a recomendar solo lo ejecutable. Aquí se agrega una exigencia: **el argumento técnico, no la conclusión**. No basta decir «HMAC no da no repudio» o «hay que usar Argon2»; hay que explicar *por qué*, de modo que el comité —y un juez— entiendan el razonamiento. La conclusión sin el porqué no aprueba.

Lo que este documento no es

No es rediseñar la firma

De H2 se pide el diagnóstico. El rediseño con firma digital y certificados es la sesión 4; no se adelanta aquí.

No es «cifrar» la contraseña

Una contraseña no se cifra ni se resume a secas: se **deriva** con factor de trabajo. Si su entrega dice «ciframos las contraseñas», está mal.

No es pedirle a la gente que la cambie

Migrar 142.000 contraseñas obligando a todos a cambiarla el mismo día no es un plan: es un incidente. Hay una solución mejor.

Cómo repartir las cinco horas

1 h

Pieza 1: diagnóstico de H1, con lo que demostró el laboratorio 3

1,5 h

Pieza 2: rediseño del almacenamiento y calibración de parámetros

1,5 h

Pieza 3: el plan de transición —la parte difícil—

1 h

Pieza 4: diagnóstico de H2, con el argumento técnico

EL ENTREGABLE

Las cuatro piezas, y qué prueba cada una

Las mismas cuatro secciones de la diapositiva de entrega. La tercera es la difícil, y es donde se ganan o se pierden los puntos.

1 Diagnóstico de H1

CONTRASEÑAS

QUÉ PIDEN	Qué algoritmo se usa, qué permite exactamente, y qué demostró el laboratorio 3 sobre la sal declarada.
QUÉ PRUEBAN	Que entienden por qué una función resumen rápida y una sal constante no protegen una contraseña, y que saben leer la discrepancia entre lo documentado y lo real.

2 Rediseño del almacenamiento

LA FUNCIÓN Y SUS PARÁMETROS

QUÉ PIDEN	La función de derivación elegida, los parámetros propuestos y cómo se calibraron. Con nombre de responsable.
QUÉ PRUEBAN	Que la elección se justifica y los parámetros se miden en hardware real, no se copian de un blog. La calibración es el corazón de la pieza.

3 Plan de transición

LA PARTE DIFÍCIL

QUÉ PIDEN	Cómo se migran 142.000 contraseñas que nadie conoce , sin pedirle a todo el mundo que cambie la suya el mismo día.
QUÉ PRUEBAN	Que conocen la solución estándar y elegante a un problema real. Hay una, y encontrarla es parte del ejercicio.

4 Diagnóstico de H2

LA FIRMA DE OPERACIONES

QUÉ PIDEN	Por qué el mecanismo actual no puede dar no repudio, con el argumento técnico, no solo la conclusión.
QUÉ PRUEBAN	Que distinguen autenticidad de no repudio, y que entienden por qué una llave compartida y simétrica no señala nunca a una persona.

PIEZA 1 · EL MÉTODO

Diagnóstico de H1

Tres cosas hay que decir, y cada una es un defecto independiente. No basta con «usan SHA-1».

Los tres defectos, y qué permite cada uno

La función es de resumen, no de derivación	SHA-1 está hecho para ser rápido. Una tarjeta gráfica prueba miles de millones de contraseñas por segundo, porque cada intento cuesta casi nada. A una función de contraseñas se le pide justo lo contrario: que cada intento cueste. H1 · Evidencia B
La sal es constante	La misma cadena para los 142.000. Eso permite una sola tabla precalculada para toda la base, y hace que dos usuarios con la misma contraseña tengan el mismo resumen —se ve quién la comparte, sin romper nada—. H1 · Evidencia B
La política es débil	Seis caracteres, sin obligación de cambiarla. Ninguna función de derivación compensa una contraseña de seis caracteres: el rediseño de la pieza 2 tiene que incluir también la política. H1

LO QUE DEMOSTRÓ EL LABORATORIO 3, Y QUE HAY QUE DOCUMENTAR AQUÍ

La documentación de la fábrica declara una sal constante, **c00pab2019**. Pero al reproducir el cálculo, los resúmenes de la muestra coinciden **sin la sal** —la condición 1 del laboratorio—. Es decir: **la sal declarada no se está aplicando**.
Escríbanlo como un hallazgo de auditoría: qué afirma la documentación, qué muestran los datos, y qué falta por comprobar (si la omisión es de toda la base o solo de la muestra, revisando el código de registro con Yeison Cardona).

EL DETALLE QUE CUESTA PUNTOS SI SE PASA POR ALTO

Que la sal *no* se aplique es **peor** que una sal constante mal elegida: sin ninguna sal, los resúmenes son directamente los de las tablas públicas precalculadas que circulan en internet, y una contraseña común aparece en su valor conocido al instante. Ese matiz —«no es que la sal sea mala, es que no existe»— es lo que separa un diagnóstico bueno de uno del montón.

PIEZA 2 · EL MÉTODO

Rediseño del almacenamiento

Una contraseña **no se cifra ni se resume a secas: se deriva** con una función pensada para eso, lenta a propósito. La pieza tiene tres decisiones y una medición.

Las tres decisiones

La función

Argon2id es la recomendada hoy; scrypt y bcrypt son alternativas aceptadas. Cualquiera de las tres, justificada. SHA-1 con más vueltas **no** es una opción.

La sal

Única y aleatoria por usuario, generada con un generador criptográfico, guardada junto al resumen. La sal no es secreta; su trabajo es que cada usuario se ataque por separado.

El «pepper» (opcional)

Un secreto del servidor, guardado aparte de la base —en un módulo de hardware o gestor de secretos—, que se suma al cálculo. Si roban la base pero no el pepper, no pueden atacar nada.

LA MEDICIÓN: CÓMO SE CALIBRAN LOS PARÁMETROS (ESTO ES LO QUE SE EVALÚA)

No hay número mágico, y copiar los de un blog es exactamente lo que la diapositiva pide no hacer. Se **mide en el hardware real** del portal: se sube el factor de trabajo —memoria e iteraciones en Argon2id, el costo en bcrypt, **N** en scrypt— hasta que un inicio de sesión tarde un objetivo tolerable para el usuario e intolerable para un ataque masivo, del orden de **250 a 500 milisegundos por contraseña**. Se reporta el número medido y el hardware en que se midió.

```
# calibrar: subir el costo hasta rozar el objetivo de tiempo, en el servidor real
import time, hashlib
t = time.perf_counter()
hashlib.scrypt(b"contrasena-de-prueba", salt=b"sal-unica-de-16-bytes", n=2**15, r=8, p=1)
print(round((time.perf_counter() - t) * 1000), "ms") # subir n hasta ~250-500 ms
```

Qué entregar en esta pieza

La función y por qué	Argon2id / scrypt / bcrypt, con una razón —no «porque sí»—.
Los parámetros medidos	El tiempo objetivo, el número al que llegaron, y el equipo donde lo midieron.
La política nueva	Longitud mínima razonable y qué se hace con las contraseñas de seis caracteres que ya existen.
El responsable	Nombre y cargo de quién queda a cargo. En el caso, Yeison Cardona escribió el portal. Parte 2 del caso

PIEZA 3 · EL MÉTODO

Plan de transición

El problema clásico: no se conocen las 142.000 contraseñas, así que no se pueden volver a derivar con la función nueva. Pedirle a todos que la cambien el mismo día no es un plan. Hay una solución estándar y elegante.

LA IDEA: DERIVAR SOBRE LO QUE YA ESTÁ GUARDADO

No hace falta conocer la contraseña para protegerla mejor. Se toma el **resumen viejo que ya está en la base** y se le aplica encima la función nueva, de una vez, para las 142.000: **nuevo = Argon2id(resumen_viejo)**. Toda la base queda protegida por Argon2id **esa misma noche**, sin que nadie escriba su contraseña.

- 1 Hoy, sin nadie conectado:** para cada usuario, calcular **Argon2id(resumen_viejo)** y guardarlo, marcando la fila como «en capas». Las 142.000 quedan protegidas de inmediato.
- 2 Cuando el usuario inicia sesión:** se aplica primero la función vieja a la contraseña que escribió, luego Argon2id, y se compara con lo guardado. Si coincide, entró bien.
- 3 En ese mismo inicio de sesión** —cuando, por única vez, sí se tiene la contraseña en claro en memoria— se recalcula el resumen «limpio» **Argon2id(contraseña)**, se reemplaza el de capas y se quita la marca. El usuario ni se entera.
- 4 Con el tiempo,** la base se va limpiando sola a medida que la gente entra. Las capas viejas desaparecen sin una migración masiva ni un día D.

POR QUÉ ESTO ES CORRECTO Y NO UN TRUCO

Es la técnica estándar de actualización de resúmenes de contraseñas: se envuelve el resumen débil dentro del fuerte. Desde la primera noche, un atacante que robe la base se enfrenta a Argon2id, no a SHA-1 pelado. Y la conversión al esquema limpio ocurre sola, sin pedirle nada a los asociados. Documenten el esquema exacto de capas —qué función vieja, con qué sal (recuerden que la declarada no se aplicaba), envuelta en qué función nueva— para que quien mantenga el portal lo entienda.

PIEZA 4 · EL MÉTODO

Diagnóstico de H2

Aquí solo se pide el diagnóstico, pero con el argumento técnico completo: por qué el mecanismo actual **no puede** dar no repudio, no importa cuántos parches se le pongan.

```
// fragmento del código de la aplicación (evidencia D)
private static final String SIGNING_KEY = "Cb4rr4-2023-HMAC-k3y";
// firma de operaciones: HMAC-SHA256(SIGNING_KEY, payload)
```

El argumento, paso a paso

Qué es HMAC	Un código de autenticación de mensajes simétrico : la misma llave crea la etiqueta y la verifica. Quien puede verificar, puede firmar. HMAC · sesión 3
Quién tiene la llave	Está incrustada en la aplicación, igual para todos y para todas las versiones desde 2023. Cualquiera que descargue la app de la tienda y la descompile la tiene. Y el servidor también. H2 · Evidencia D
Qué exige el no repudio	Que solo el titular pueda producir la firma. Con una llave que 142.000 personas —y el servidor— comparten, cualquier operación pudo haberla hecho cualquiera. No señala a nadie. Ley 527, art. 28
La conclusión	No es que esté «mal configurado»: es que un MAC simétrico no puede dar no repudio por diseño. Se necesita una firma asimétrica , con una llave privada bajo control exclusivo del titular. Eso es la sesión 4.

LA DISTINCIÓN QUE EL COMITÉ —Y EL JUEZ— NECESITAN

HMAC sí da **autenticidad e integridad**: prueba que el mensaje viene de alguien que conoce la llave y que no se alteró. Lo que no da es **no repudio**: no prueba *quién*, porque la llave no es de una persona. El apoderado que hizo la pregunta que nadie supo responder necesitaba lo segundo, y el mecanismo actual solo entrega lo primero. Esa es, en una línea, la raíz de las treinta y ocho reclamaciones.

MÉTODO

De dónde sale cada dato

La entrega se arma con el caso, sus evidencias, el laboratorio 3 y lo que vieron en la sesión 3.

FUENTE	QUÉ ENCUENTRAN AHÍ
Hallazgo H1 del caso	El SHA-1 con sal constante, sin derivación ni factor de trabajo, y la política de seis caracteres. Insumo de las piezas 1, 2 y 3.
Evidencia B Parte 8 del caso	La muestra de la tabla de usuarios: los tres resúmenes, dos iguales, y la sal declarada. Es lo que se analiza en la pieza 1.
Laboratorio 3	La demostración de que los resúmenes se reproducen sin la sal declarada. Es la prueba de la discrepancia de la pieza 1.
Hallazgo H2 y Evidencia D del caso	La firma HMAC-SHA256 con la llave compartida Cb4rr4-2023-HMAC-k3y , extraíble de la app. Es la pieza 4.
Sesión 3	Funciones resumen, por qué MD5 y SHA-1 murieron, la diferencia entre integridad y autenticidad con HMAC, y por qué una contraseña se deriva con factor de trabajo. El marco entero.
Ley 527 de 1999 art. 28	Los atributos de una firma confiable, entre ellos que esté bajo control exclusivo del firmante. Es el respaldo legal del diagnóstico de H2.

DOS ERRORES QUE HUNDEN LA ENTREGA

Confundir cifrar con derivar. «Ciframos las contraseñas con AES» es un error de concepto: una contraseña no se cifra —se podría descifrar—, se deriva con una función de un solo sentido y factor de trabajo. **Confundir autenticidad con no repudio.** Decir que HMAC «no es seguro» pierde el punto: es seguro para lo que hace —autenticar—, solo que eso no es lo que el caso necesita. La precisión en estas dos distinciones es la mitad de la nota.

EVALUACIÓN

Con qué criterio se califica

Se evalúa el razonamiento. La pieza 3 pesa más porque es la que separa a quien entendió el problema de quien copió una receta.

CRITERIO	PESO	QUÉ SE MIRA
Diagnóstico de H1 (pieza 1)	20 %	Los tres defectos, y la discrepancia de la sal documentada como hallazgo, no como conclusión suelta.
Rediseño y calibración (pieza 2)	25 %	Función justificada, sal única, y parámetros medidos con su tiempo objetivo y su hardware. No copiados.
Plan de transición (pieza 3)	30 %	La migración por capas, correcta y explicada, sin pedirle a nadie que cambie su contraseña el mismo día.
Diagnóstico de H2 (pieza 4)	25 %	El argumento técnico completo de por qué un MAC simétrico no puede dar no repudio, no solo la conclusión.
Total	100 %	Parte 3 de las seis que componen el informe de aseguramiento criptográfico

Antes de entregar

☐ La pieza 1 nombra los **tres** defectos de H1, no solo «usan SHA-1».

☐ La discrepancia de la sal está escrita como **hallazgo de auditoría** : qué se declara, qué muestran los datos, qué falta comprobar.

☐ La pieza 2 elige una **función de derivación** (no SHA con más vueltas), con **sal única por usuario** y **parámetros medidos** con su tiempo y su hardware.

☐ La pieza 3 migra por **capas** sin conocer las contraseñas y sin un día D, y documenta el esquema exacto.

☐ La pieza 4 explica **por qué** un MAC simétrico no da no repudio, con el argumento, no solo la conclusión.

☐ No se confunde **cifrar con derivar** ni **autenticidad con no repudio** .

☐ El almacenamiento tiene un **responsable con nombre y cargo** , y el archivo lleva el nombre del equipo y sus integrantes.

PLANTILLA

Para armar el documento

Equipo: _____ Fecha: ____ / ____

Integrantes: _____

PIEZA 1 · DIAGNÓSTICO DE H1

DEFECTO	QUÉ PERMITE EXACTAMENTE
La función (SHA-1)	
La sal constante / ausente	
La política	

PIEZA 2 · REDISEÑO

DECISIÓN	ELECCIÓN	JUSTIFICACIÓN / MEDICIÓN
Función de derivación		
Parámetros (medidos)		
Sal · política · responsable		

PIEZAS 3 Y 4 · TRANSICIÓN Y DIAGNÓSTICO DE H2

PLAN DE TRANSICIÓN (ESQUEMA DE CAPAS)	DIAGNÓSTICO DE H2 (POR QUÉ NO HAY NO REPUDIO)

RECUERDEN

La entrega es un documento del equipo con las cuatro piezas, al inicio de la sesión 4. La que más pesa es la 3 —la migración—, y la que más se confunde es la 4: HMAC no es «inseguro», es que autenticar no es lo mismo que probar quién.